TECHNICAL DOCUMENTATION

Sinau Agentic Architecture & Flow

A comprehensive guide to the agentic AI research platform: ReAct agent loop, nine-tool ecosystem, two-tier memory, RAG pipeline, and real-time SSE streaming between Flask and React.

Joskoding
v2.0 · 2026.06
Ollama + Flask + React + Vite

Contents

01 · EXECUTIVE SUMMARY

Executive Summary

Sinau Agentic is a teaching platform that demonstrates how agentic AI systems work. It implements the ReAct pattern in a research assistant that can search the web, query Indonesian academic databases (Garuda), read PDFs, and synthesize answers — all through a local Ollama LLM with streaming real-time feedback to a React frontend.

Key Takeaways

- The agent runs a ReAct loop (Reason + Act) for up to 7 steps per query, calling tools between LLM invocations.
- Nine tools form the ecosystem: web search, Garuda academic search (with parallel multi-keyword), PDF reading, webpage reading, note management, and internal thinking.
- A two-tier memory system (conversation history + working memory) combined with cosine-similarity RAG via bge-m3 embeddings provides persistent context.
- Real-time communication flows through Server-Sent Events: Flask yields typed event dicts; the React frontend consumes them via Fetch ReadableStream.

QUESTIONS THIS DOCUMENT ANSWERS

How does the ReAct loop orchestrate tool calls and final answers? What is the full architecture from browser to LLM? How do SSE events carry tool status and results to the UI? How does Garuda academic search with parallel multi-keyword queries work? What state does the agent preserve between user queries?

02 · SYSTEM ARCHITECTURE

System Architecture

The platform spans five layers: a React browser client, a Vite dev proxy, a threaded Flask server, the agentic Python runtime, and a local Ollama instance running two models — gemma4 for chat and bge-m3 for embeddings.

Five-Layer Stack

Each layer has a distinct responsibility. The Agent Core is the focal layer where the ReAct loop, tool execution, and memory management live. Layers above carry presentation concerns; layers below carry infrastructure and model serving.

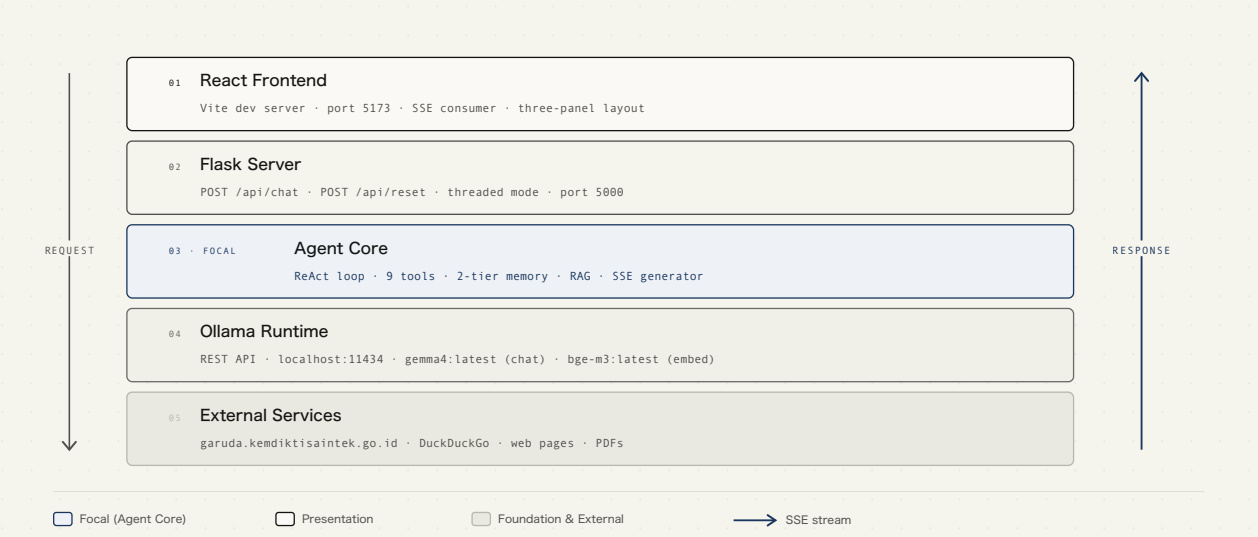

Figure 1. The five-layer architecture with the Agent Core as the focal layer. Request flow descends through Vite proxy to Flask; SSE events stream back up through the same path.

Technology Choices

Component	Technology	Rationale
LLM	Ollama + gemma4:latest	Local, no API keys, fast tool-calling support
Embeddings	Ollama + bge-m3:latest	Multilingual, strong on Indonesian-English mixed text
Backend	Flask 3.0 (threaded)	Lightweight, native SSE generator support
Frontend	Vite + React 19	Fast HMR, API proxy to Flask, no SSR needed
Streaming	Server-Sent Events	Unidirectional server-to-client; simpler than WebSocket for this use case
Web Search	DuckDuckGo (ddgs)	No API key, privacy-respecting, returns text snippets
PDF Reading	pypdf	Pure Python, extracts text + references from academic papers

03 · THE REACT AGENT LOOP

The ReAct Agent Loop

The agent uses the ReAct pattern: it reasons about what to do next, acts by calling a tool, observes the result, and feeds that observation back into its reasoning. This cycle repeats until the LLM produces a final answer or hits the 7-step cap.

Loop Mechanics

Each iteration sends the full conversation history (system prompt + user message + all prior tool calls and results) to Ollama. The LLM returns one of two outcomes: a set of `tool_calls` (the agent must execute tools and loop back) or a `content` string (the final answer). The loop enforces a max of 7 steps; if the LLM still calls tools at step 7, a forced-final-answer prompt is injected.

> The agent never decides which tool to call next — the LLM does. The agent's job is to execute whatever the model requests, format the result, and feed it back. This is the essence of the ReAct pattern: the LLM is the brain; the agent is the hands.

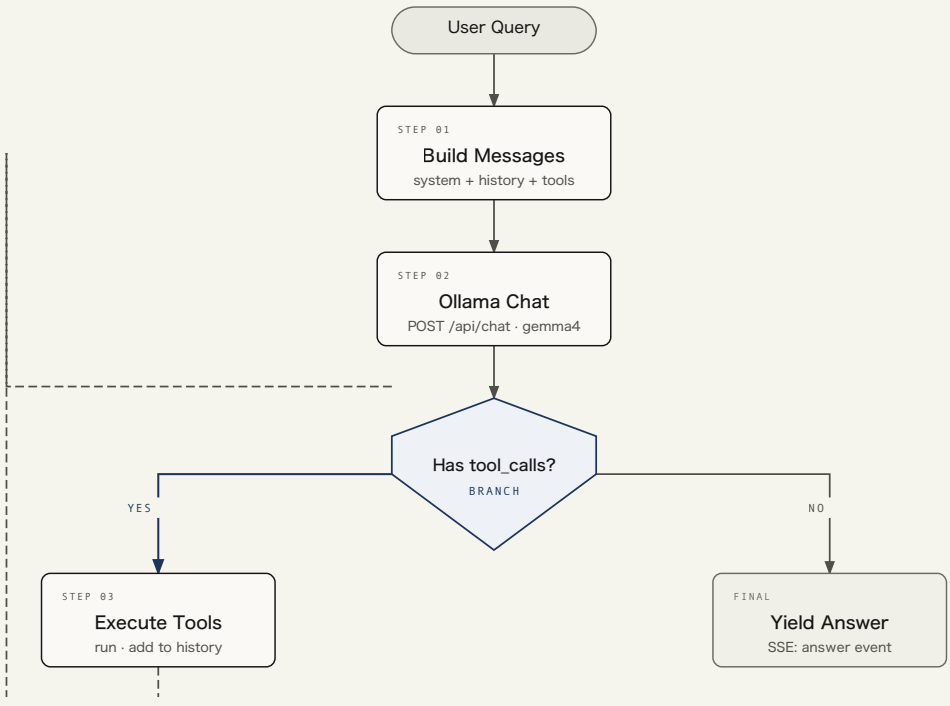

Figure 2. The ReAct loop with the tool-call decision as the focal branch. When tool_calls are present, the agent executes them and feeds results back into the next iteration. When absent, the content is yielded as the final SSE answer event.

Key Implementation Details

The loop lives in `AgenticResearch.research_stream()` (`agent.py:102`), a Python generator that yields typed event dicts. Two versions exist: `research()` for CLI output and `research_stream()` for SSE streaming. Both share the same core logic but differ in how they emit progress.

```
def research_stream(self, query):
    yield {"type": "user", "content": query}
    self.memory.add_user_message(query)

    for step in range(1, self.max_steps + 1):
        yield {"type": "step", "step": step, "max": self.max_steps}
        messages = self.memory.build_messages()
        response = chat(messages, tools=self.tools)

        if response.get("tool_calls"):
            for tc in response["tool_calls"]:
                func = tc["function"]
                tool_name = func["name"]
                tool_args = func.get("arguments", {})
                yield {"type": "tool_call", "name": tool_name, "args": tool_args}
                result = execute_tool(tool_name, tool_args)
                yield {"type": "tool_result", "name": tool_name, "result":
result[:800]}
                self.memory.add_tool_call(tool_name, tool_args)
                self.memory.add_tool_result(tool_name, result)
            continue  # loop back to LLM

        answer = (response.get("content") or "").strip()
        if not answer:
            # retry once with explicit prompt
            ...
        self.memory.add_assistant_message(answer)
        yield {"type": "answer", "content": answer}
        return
```

04 · TOOL ECOSYSTEM

Tool Ecosystem & Garuda Integration

Nine tools form the agent's action repertoire. Each is defined with an OpenAI-compatible JSON schema in `tools.py` and dispatched through a unified `execute_tool()` function. The Garuda academic search tools are the platform's differentiator, providing access to Indonesia's national research database.

Tool Inventory

#	Tool	Source	Description
1	web_search	DuckDuckGo	General web search returning title + snippet + URL
2	read_webpage	requests + BeautifulSoup	Fetch and extract text content from any URL
3	garuda_search	garuda.kemdiktisaintek.go.id	Single-keyword search of Indonesian academic publications
4	garuda_multi_search	Garuda (parallel)	Multi-keyword parallel search via ThreadPoolExecutor; 3–5 queries in one call
5	garuda_detail	Garuda detail page	Full metadata: authors, journal, DOI, abstract, download link
6	garuda_read_pdf	pypdf + HTTP download	Download PDF, extract body text + references (daftar pustaka)
7	save_note	Working memory	Store a key-value fact for later recall
8	recall_note / list_notes	Working memory	Retrieve or enumerate stored facts
9	think	Internal	Scratchpad for the LLM to plan between tool calls

Garuda Multi-Search

The most architecturally interesting tool. Instead of a single search, `garuda_multi_search` accepts an array of 3–5 keyword variations and executes them in parallel via `concurrent.futures.ThreadPoolExecutor`. Results are deduplicated by Garuda ID and returned as a unified, ranked list. This approach dramatically improves recall for Indonesian-language queries where keyword variance is high.

> The system prompt instructs the LLM to break user queries into specific keyword angles: main topic, location/context, impact/effect, and synonyms in both English and Indonesian. A query like “renewable energy in Dieng” becomes [“renewable energy Dieng”, “energi terbarukan Dieng”, “dampak geothermal Dieng”, “renewable energy impact highland”].

Paper Parsing Pipeline

Results from Garuda searches are parsed into structured paper objects via `_parse_papers_from_result()` (`agent.py:167`). A regex extracts index, title, authors, journal, DOI, and Garuda ID from the formatted search output. These structured objects flow through SSE `tool_result` events to populate the frontend Papers panel.

```
papers.append({
    "index":     m.group(1),
    "title":     m.group(2).strip(),
    "authors":   m.group(3).strip(),
    "journal":   (m.group(4) or "").strip(),
    "doi":       (m.group(5) or "").strip(),
    "garuda_id": m.group(6).strip(),
})
```

05 · MEMORY & RAG

Memory, RAG & State Management

The agent maintains state across two dimensions: conversation history (short-term, per-session) and RAG-backed knowledge retrieval (semantic, cosine-similarity search over embedded notes). A third layer, session isolation, keeps multi-user state clean in the Flask server.

Two-Tier Memory

The `AgentMemory` class (`memory.py`) manages two distinct stores:

Tier	Store	Persistence	Use
1. Conversation History	`self.history` (list of message dicts)	Session lifetime	Full message log: user, assistant, tool_call, tool_result roles
2. Working Memory	`self.working` (dict key-value)	Session lifetime	Facts saved via `save_note` , recalled via `recall_note`

The `build_messages()` method prepends the system prompt to the full history before sending to Ollama. Every tool call and its result is recorded as a structured message, giving the LLM full context of what has been explored so far.

RAG Pipeline

When the agent calls `save_note` , the note is also embedded via bge-m3:latest and stored as a numpy vector in the `RAG` class (`rag.py`). On `rag_query()` , the query text is embedded and cosine similarity is computed against all stored vectors to return the `top_k` most relevant documents.

```
class RAG:
    def __init__(self):
        self.documents = []
        self.embeddings = None  # numpy array

    def add(self, text):
        self.documents.append(text)
        emb = embed([text])[0]
        if self.embeddings is None:
            self.embeddings = np.array([emb])
        else:
            self.embeddings = np.vstack([self.embeddings, emb])

    def query(self, query_text, top_k=3):
        q_emb = embed([query_text])[0]
        similarities = np.dot(self.embeddings, q_emb)
        top_indices = np.argsort(similarities)[-top_k:][::-1]
        return [self.documents[i] for i in top_indices]
```

Session Isolation

Flask's `server.py` maintains an in-memory `sessions` dict mapping session IDs to `AgenticResearch` instances. Each new session (triggered by "New Chat" in the frontend) calls `POST /api/reset` to clear the agent from the dict, then creates a fresh instance on the next `/api/chat` call. Sessions are scoped per browser session; there is no cross-session persistence.

06 · SSE STREAMING PIPELINE

SSE Streaming Pipeline

Real-time communication between the Flask backend and React frontend runs over Server-Sent Events. The agent's generator yields typed event dicts; Flask wraps them in SSE `data:` lines; the React frontend consumes them via the Fetch API's `ReadableStream` reader.

Backend: Generator to SSE

The `POST /api/chat` endpoint (`server.py:20`) creates an inner generator that iterates over `agent.research_stream(query)` , serializes each event dict as JSON, and wraps it in the SSE wire format:

```
def generate():
    for event in agent.research_stream(query):
        yield f"data: {json.dumps(event)}\n\n"
    yield 'data: {"type": "done"}\n\n'

return Response(generate(), mimetype="text/event-stream")
```

Event Type Reference

Event	Payload	When Emitted	Frontend Action
`user`	`{type, content}`	Query received	(internal)
`step`	`{type, step, max}`	Each loop iteration	Updates status bar step counter
`tool_call`	`{type, name, args}`	LLM requests a tool	Adds running badge to tool status bar
`tool_result`	`{type, name, result, papers}`	Tool completes	Marks badge done; pushes papers to panel
`answer`	`{type, content}`	Final LLM response	Appends agent message to chat
`done`	`{type: "done"}`	Stream end marker	(internal; triggers `finally` cleanup)

Frontend: SSE Consumption

The SSE spec's native `EventSource` API only supports GET requests. Since the agent needs POST (to send the query and session ID in the body), the frontend uses `fetch()` with a `ReadableStream` reader instead:

```
const res = await fetch('/api/chat', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ query, session: String(activeId) }),
})
const reader = res.body.getReader()
const decoder = new TextDecoder()
let buffer = ''

while (true) {
  const { done, value } = await reader.read()
  if (done) break
  buffer += decoder.decode(value, { stream: true })
  const lines = buffer.split('\n\n')
  buffer = lines.pop() || ''

  for (const line of lines) {
    if (!line.startsWith('data: ')) continue
    const data = JSON.parse(line.slice(6))
    // dispatch by data.type → React setState
  }
}
```

> The `buffer` variable is essential: SSE chunks can arrive mid-event. The code accumulates partial data across `read()` calls, splits on `\n\n` (the SSE event delimiter), and keeps the trailing partial chunk in `buffer` for the next iteration. Without this, partial JSON would throw parse errors on every chunk boundary.

Vite Proxy

The Vite dev server proxies `/api` requests to `http://127.0.0.1:5000` (configured in `vite.config.js`). This avoids CORS issues in development and keeps the frontend and backend on the same origin from the browser's perspective.

07 · FRONTEND ARCHITECTURE

Frontend Architecture

The React frontend (`App.jsx` , 463 lines) is a single-page application with three-panel layout: a session sidebar, a chat area with streaming indicators, and a papers panel. It consumes SSE events via Fetch ReadableStream and manages multi-session state entirely in React hooks.

Component Structure

The entire app is a single `App` component with no child components. State is managed through seven `useState` hooks and three `useRef` hooks for real-time sync during streaming:

State	Type	Purpose
`sessions`	Array of session objects	All chat sessions: each has id, title, messages[], papers[]
`activeId`	Number (timestamp)	Currently selected session
`input`	String	Controlled input value with @mention support
`streaming`	Boolean	SSE stream active flag (disables input, shows dots)
`toolStatus`	Array of {name, state, key}	Tool execution badges: running → done
`step`	{current, max}	Current ReAct iteration for status bar
`showMention / mentionIndex / mentionItems`	@ autocomplete state	Dropdown for paper mentions and followups

Three-Panel Layout

Left: Session Sidebar (240px)

Lists all sessions by title. Each session stores its own messages and papers independently. The “New Chat” button calls `POST /api/reset` to clear the backend session, then creates a new frontend session entry. Switching sessions resets tool status and step indicators.

Center: Chat Area

Messages render as user/agent pairs with avatar circles (“U” / “AI”). User messages use a right-aligned bubble with `border-radius: 12px 12px 4px 12px` . Agent messages split content by `\n` into `<p>`

tags. During streaming, a loading indicator shows three animated dots next to the "AI" avatar, which disappears once the `answer` SSE event arrives. Four followup suggestion buttons appear below the last agent message.

> Performance note: the `send` function is wrapped in `useCallback(fn, [])` to avoid recreating the event handler on every keystroke. Refs mirror `input`, `streaming`, and `activeId` state so the callback always reads the latest values without depending on React state.

Right: Papers Panel (320px)

Displays papers parsed from Garuda search results. Each paper card links to `garuda.kemdiktisaintek.go.id/documents/detail/{garuda_id}`. Papers are deduplicated by Garuda ID as they arrive through `tool_result` events. The @mention autocomplete scans this panel's papers for inline references.

@Mention System

Typing `@` in the input triggers an autocomplete dropdown with three categories of suggestions: papers from the current session (filtered by title/author match), an "Elaborate" action, and four pre-defined follow-up questions. Selection replaces the `@query` text with the chosen item. Navigation uses ArrowUp/ArrowDown/Tab/Enter/Escape.

Styling

The UI uses a Cal.com-inspired design system defined as CSS custom properties in `index.css`: warm parchment backgrounds (`--canvas`), card surfaces (`--surface-card`), and ink-dark accents (`--ink`). Tailwind CSS 4 provides utility classes; custom animations (`fade-up`, `pulse-dot`) handle message entrance and loading indicators.

KEY DESIGN DECISIONS

Single-component architecture avoids prop-drilling overhead for a three-panel app. Refs-for-streaming pattern keeps the SSE callback stable across re-renders. Papers deduplication by Garuda ID prevents duplicate entries when multiple searches return overlapping results.